Prompt 1: Project Setup, Data Engine & State Management

> "Act as an expert Android developer. Help me set up the initial Kotlin architecture for an Android app called **VangtiChai** with a minimum API level of 21 (Android 5.0 Lollipop).
>
> Please write the code and step-by-step setup for:
>
> 1. **Change Calculation Logic**: A helper/utility class or function that takes an integer amount in Bangladeshi Taka and returns a breakdown of notes for denominations: **500, 100, 50, 20, 10, 5, 2, and 1**.
> 2. **State & Lifecycle Management**: A `ViewModel` (or `onSaveInstanceState` handling) that holds the currently entered amount string and note breakdown map/list so that the state persists seamlessly across orientation changes (Portrait <-> Landscape) and activity recreation.
> 3. **Input Handling Logic**: Functions to append digits (0–9) to the right of the current string and a clear function that resets the amount to 0/empty.
> 4. `MainActivity.kt` setup connecting the ViewModel state to UI update methods."

Your task is to create an Android app (VangtiChai) that lets you enter a money amount via a numeric keypad implemented from scratch (to practice layouts!), and calculates change for that amount in terms of Taka Notes (500,100,50,20,10,5,2,1). You can make your own choice between Kotlin, Java and Flutter, but you are welcome to try Kotlin on your own initiative), and for minimum API level, you can set "API 21: Android 5.0 (Lollipop)" (or lower if need be). Here is what you have to implement: 1. In portrait mode, your app's main view shows a numeric keyboard consisting of buttons for the 10 digits and a clear button in the configuration shown below. To the left of it, there should be a table listing for the notes that make up the change for any entered amount. In the top part of the screen, the entered amount is shown, next to a "Taka:" label. 2. When you tap any button, the corresponding digit is added to your current amount from the right (this is to make entry as easy as possible for you to program). I.e. if you type the digits '2', '3', and '4' in this order, you first will see an amount of 2, then 23, and then 234. 3. When turning your device to landscape mode, the current interface should save state (keep the currently entered amount and change list), and present a slightly altered layout: 4. The layouts on your app should work gracefully on phones and tablets. Test

with the emulator on at least a Pixel XL phone (see screenshots above), and a Nexus 10 tablet (see screenshots below).

Prompt 2: Resource Management (`dimens.xml`, `strings.xml`, `colors.xml`)

> "Now, set up the externalized resource files for **VangtiChai** according to strict Android best practices. No values should be hardcoded in XML layout files.
>
> Please generate:
>
> 1. `res/values/dimens.xml`: Define text sizes, margins, key heights/widths, and paddings suitable for standard phone displays.
> 2. `res/values-sw600dp/dimens.xml`: Define scaled dimension values for tablet displays (e.g., Nexus 10 / 800x1280dp screens).
> 3. `res/values/strings.xml`: Define all UI strings, including labels ('Taka:', note denominations, 'Clear', button labels, app name).
> 4. `res/values/colors.xml`: Define basic clean styling colors for buttons, text, background, and grid borders."

Prompt 3: Phone Layouts (`ConstraintLayout` for Portrait & Landscape)

> "Create the phone layout files using **ConstraintLayout** for the `MainActivity`. Use `@dimen/` and `@string/` references for all attributes.
>
> Please generate:
>
> 1. **Portrait Layout (`res/layout/activity_main.xml`)**:
>   - **Top**: Text header showing 'Taka:' label next to the currently entered amount.
>   - **Left/Main area**: A Table Layout or structured view listing the note breakdown (500 down to 1 Taka).
>   - **Bottom/Right**: A custom numeric keypad grid using `ConstraintLayout` with buttons for digits `0`–`9` and a `Clear` button. Use ConstraintLayout chains or guidelines to build the keypad grid.
> 2. **Landscape Layout (`res/layout-land/activity_main.xml`)**:

    - Re-architect the same UI elements into a landscape-optimized `ConstraintLayout` where the numeric keypad, note breakdown table, and display header sit side-by-side or balanced gracefully across the wider horizontal space.
3. Connect click listeners in `MainActivity` for all keypad buttons (0–9 and Clear) to update the ViewModel and sync the view."

Prompt 4: Tablet Qualifier Layouts (`sw600dp` for Portrait & Landscape)

"Now generate the tablet-specific layout XML files using the `sw600dp` resource qualifier to support larger screens like the **Nexus 10 tablet (800x1280dp)**.

Please generate:

1. `res/layout-sw600dp/activity_main.xml` (Tablet Portrait): Uses larger dimension references (`@dimen/...`), proportional keypad sizing, and balanced padding so the UI does not stretch awkwardly across a 10-inch screen.
2. `res/layout-sw600dp-land/activity_main.xml` (Tablet Landscape): Optimized layout side-by-side grid arrangement utilizing the high horizontal real estate.
3. Ensure all 4 layout variations share identical View IDs (`@+id/...`) so `MainActivity.kt` works seamlessly across all configurations without runtime crashes or missing view references."

Prompt 5: Verification, Edge Cases & README Documentation

"Let's finish and polish the **VangtiChai** project:

1. Review edge cases: Handle integer overflow when entering very long digit strings, leading zeros handling (e.g., entering '007' should become '7'), and empty state display.
2. Provide a testing verification checklist for emulator devices: **Pixel XL phone (411x731dp)** and **Nexus 10 tablet (800x1280dp)** in both orientations.
3. Draft a clean `README.txt` file summarizing:
    - Project overview & features.

- Architecture overview (ConstraintLayout usage & state management).
- List of tested devices and screen sizes (Pixel XL, Nexus 10, plus other target screen profiles).
- Instructions on building and running the application."